BCSE307P · COMPILER DESIGN LAB

ICG WITH BACKPATCHING

One-Pass Intermediate Code Generation for
Control-Flow Statements using Flex and Bison

Review 2 and Review 3 Report

Progress, Implementation and Testing

Team 15

Prithvi Raghu	24BCE2624
Jai Narayan Jayaprakash Jayanthi	24BCE2751
Divyanshu Pal	24BCE2856

Faculty: Dr. Naga Priyadarsini

October 2026

Contents

1 Introduction **1**

1.1 About this report 1

1.2 Problem recap 1

1.3 Status at Review 1 1

1.4 Plan for Reviews 2 and 3 2

2 Review 2: Conditionals, while and Boolean Operators **3**

2.1 if-else and the dangling else 3

2.2 while loops 4

2.3 Short-circuit boolean conditions 5

2.4 Nesting 6

2.5 Error reporting 7

2.6 Trace mode 8

2.7 Review 2 outcome 9

3 Review 3: Remaining Loops, Loop Exits and Comparison **10**

3.1 do-while 10

3.2 for 10

3.3 break and continue 11

3.4 Step mode and the remaining outputs 13

3.5 Two-pass baseline 14

3.6 Comparison of the two approaches 15

3.7 Documentation 17

4 Testing and Validation **18**

4.1 Build 18

4.2 Regression suite 18

4.3 Stress programs 20

4.4 Error cases 21

4.5 Randomised comparison with the baseline 21

5 Deviations from the Plan and Problems Solved **23**

5.1 Trace viewer replaced by step mode 23

5.2 Position of the N marker 23

5.3 Exits from loops 23

5.4 Line numbers for split statements 23

5.5 Grammar warnings 23

5.6 Guard against wrong actions 24

6 Current Status and Remaining Work **24**

6.1 Status 24

6.2 Remaining work and possible extensions 24

References **25**

Appendix A. Reproducing the Results **26**

List of Figures

Figure 2.1	An else-if ladder with compound conditions (examples/e1_ladder.src)	4
Figure 2.2	Negated disjunction inside a conjunction (examples/e2_bool.src)	6
Figure 2.3	Syntax, semantic and lexical errors with line numbers and exit codes	7
Figure 2.4	Trace of a while loop with a break (examples/e5_trace.src)	8
Figure 3.1	Three nested loop kinds with break and continue at different depths (examples/e3_deep.src)	12
Figure 3.2	Symbol table and quadruple table for a bubble-sort skeleton (examples/e4_sort.src)	14
Figure 3.3	The baseline's code before label resolution, and diffs against icg	15
Figure 3.4	Statistics from both generators on the four stress programs	16
Figure 4.1	Clean build of both generators	18
Figure 4.2	Full regression run: 35 of 35 checks pass	20
Figure 4.3	Randomised differential test on 500 programs	22

List of Tables

Table 1.1	Position at the end of Review 1	1
Table 1.2	Schedule presented at Review 1	2
Table 1.3	Planned items and their status	2
Table 2.1	Errors detected by the generator	7
Table 2.2	Life of each jump in Figure 2.4	9
Table 3.1	Layout of for (i = 0; i < n; i = i + 1) s = s + i; (test 09_for)	11
Table 3.2	Loop exits in Figure 3.1, checked by hand	12
Table 3.3	Command-line options of icg	13
Table 3.4	icg versus twopass on regression tests and stress programs	16
Table 4.1	Regression tests	19
Table 4.2	Stress programs	21
Table 4.3	Error programs (Figure 2.3 and the regression suite)	21
Table 6.1	Project status after Review 3 work	24

1 Introduction

1.1 About this report

This report documents the work carried out on the project *ICG with Backpatching* between Review 1 (9 October 2026) and Review 3 (23 October 2026). Review 1 delivered the lexer, the full grammar, the quadruple table and the patch-list helpers, with code generation working for assignments and the plain `if` statement. The plan presented at that review divided the remaining work into two phases. This report takes each phase in turn, describes what was implemented and how, and shows the generator running on the regression tests and on a set of harder programs written specifically for this report.

Every screenshot in the report is a terminal session in which the commands shown were actually run against the current source, and every output was checked by hand against the translation scheme before being included.

1.2 Problem recap

A syntax-directed translator that emits three-address code while it parses reaches jumps whose targets have not been generated yet. In `if (a < b) y = c;` the jump taken when the condition is false must land after `y = c`, but at the moment the condition is reduced that quadruple does not exist. Backpatching (Aho et al. [1], sec. 6.7) solves this by emitting the jump with an empty target, recording its number in a list, and filling every jump on the list once the target is known. The project implements this scheme in a single pass using Flex and Bison, with no intermediate tree and no label-resolution pass.

1.3 Status at Review 1

Table 1.1 Position at the end of Review 1

Component	State on 9 October
Lexer (`lexer.l`)	Complete for the full language, including loop keywords and boolean operators
Grammar (`parser.y`)	Complete and conflict-free; actions written for assignment, arithmetic, relational conditions and `if`
Quad table, patch lists, symbol table	Implemented: `emit`, `newtemp`, `makelist`, `merge`, `backpatch`, `symtab_insert`
Tests	3 of 3 passing (`01_assign`, `02_if`, `08_syntax_error`)
Parsed but not yet translated	`if-else`, `while`, `&&`, `||`, `!`, `true`, `false`

1.4 Plan for Reviews 2 and 3

Table 1.2 reproduces the schedule presented at Review 1, and Table 1.3 lists each planned item against what was delivered. All planned items were completed. One item, the separate HTML trace viewer, was delivered in a different form; the reason is given in Section 5.1.

Table 1.2 Schedule presented at Review 1

Milestone	Date	Scope
Review 1	9 Oct 2026	Lexer, full grammar, quad table, patch lists; assignment and `if`
Review 2	16 Oct 2026	`if-else`, `while`, `&& || !`, nesting, error reporting; `--trace` mode
Review 3	23 Oct 2026	`do-while`, `for`, `break` / `continue`; trace viewer; two-pass comparison; manuscript
Submission	30 Oct 2026	Final report and presentation uploaded to VTOP

Table 1.3 Planned items and their status

Review	Planned item	Status	Where described
2	`if-else` with dangling-else resolution	Done	Section 2.1, Figure 2.1
2	`while` loop	Done	Section 2.2
2	Short-circuit `&&`, `||`, `!`, `true`, `false`	Done	Section 2.3, Figure 2.2
2	Nesting to any depth	Done	Section 2.4
2	Error reporting with line numbers	Done	Section 2.5, Figure 2.3
2	`--trace` mode	Done	Section 2.6, Figure 2.4
3	`do-while` loop	Done	Section 3.1
3	`for` loop	Done	Section 3.2
3	`break` and `continue`	Done	Section 3.3, Figure 3.1
3	Trace viewer	Done, as `--step`	Sections 3.4 and 5.1
3	Two-pass baseline and comparison	Done	Sections 3.5, 3.6, Figures 3.3, 3.4
3	Manuscript (report and slides)	Done	Section 3.7

2 Review 2: Conditionals, while and Boolean Operators

Review 2 extended code generation from the single `if` statement to the rest of the structured control flow that does not need loop exits: `if-else`, `while`, compound conditions and arbitrary nesting. It also added error reporting and the trace mode, which is the project's main addition to the textbook scheme.

2.1 if-else and the dangling else

An `if-else` needs three kinds of jump: the true exits of the condition go to the then-branch, the false exits go to the else-branch, and the end of the then-branch must jump over the else-branch. The last jump is produced by the marker non-terminal `N`, which emits an unfilled `goto` and returns it as a one-element list.

```
S → if ( B ) M1 S1 else N M2 S2
      backpatch(B.truelist,  M1.instr)
      backpatch(B.falselist, M2.instr)
      S.nextlist = merge(S1.nextlist, N.nextlist, S2.nextlist)

N → ε     N.nextlist = makelist(nextinstr);  emit("goto _")
```

The grammar for `if` and `if-else` is ambiguous when one `else` follows two `if`s. Bison is told to prefer shifting `else` through the precedence declarations `%precedence LOWER_THAN_ELSE` and `%precedence ELSE`, so an `else` binds to the nearest `if`. The textbook places `N` before `else`; in Bison that produced a reduce/reduce conflict, which precedence cannot settle. Moving `N` after `else` turns it into a shift/reduce choice, and the emitted `goto` still lands directly after the then-branch because nothing is emitted while `else` is shifted. `%expect 0` makes the build fail if any conflict reappears.

Figure 2.1 runs an else-if ladder of the kind found in a grading program. Each `else if` is simply an `if-else` nested in the else-branch of the previous one, so the ladder exercises the rule four levels deep.

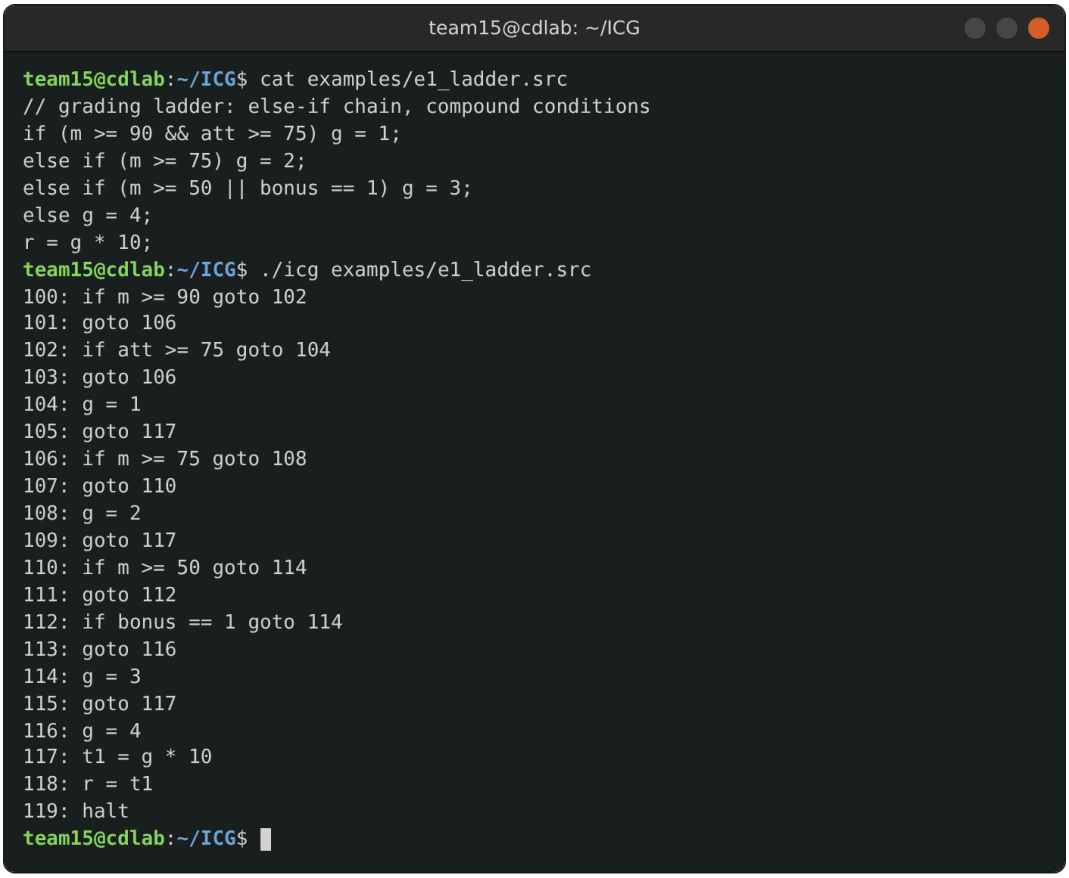


Figure 2.1 An else-if ladder with compound conditions (`examples/e1_ladder.src`)

The three jumps over the remaining branches, quads 105, 109 and 115, are all `N` jumps. None of them can be filled while the ladder is being parsed, because their target is the statement after the whole ladder. They are merged into the ladder's `nextlist` and filled together with 117 when `r = g * 10` is reached. In the third condition, `m >= 50 || bonus == 1`, a true `m >= 50` jumps straight to 114 and the second test is skipped. Quad 111 jumps to the very next quad; this is the standard code shape for `||` and is left as it is, since no optimisation is done (Section 6.2).

2.2 while loops

A `while` loop needs one backward jump, from the end of the body to the test, and that target is already known when the jump is emitted. Such jumps are emitted with `emit_goto`, which writes the target immediately and never puts the jump on a patch list. The forward jumps, the false exits of the condition, become the loop's `nextlist`.

```
S → while M1 ( B ) W S1
      backpatch(S1.nextlist, M1.instr)     // end of body returns to the test
      backpatch(B.truelist,  W.instr)      // true exits enter the body
      emit_goto(M1.instr)
      S.nextlist = merge(B.falselist, loop.breaks)
```

`W` is a variant of `M` that also opens a new entry on a loop stack, where the `break` and `continue` jumps of the body are collected (Section 3.3). The output for `tests/04_while.src` is shown below.

```
i = 0;                          100: i = 0
while (i < n) i = i + 1;        101: if i < n goto 103
done = 1;                       102: goto 106
                                103: t1 = i + 1
                                104: i = t1
                                105: goto 101
                                106: done = 1
                                107: halt
```

2.3 Short-circuit boolean conditions

Boolean operators generate no arithmetic. Each one only rearranges the true and false lists of its operands, so the right operand is evaluated only when the left one has not already decided the result.

```
B → B1 || M B2    backpatch(B1.falselist, M.instr)
                  B.truelist  = merge(B1.truelist, B2.truelist)
                  B.falselist = B2.falselist
B → B1 && M B2    backpatch(B1.truelist, M.instr)
                  B.truelist  = B2.truelist
                  B.falselist = merge(B1.falselist, B2.falselist)
B → ! B1          B.truelist = B1.falselist;  B.falselist = B1.truelist
B → true          B.truelist  = makelist(nextinstr); emit("goto _")
B → false         B.falselist = makelist(nextinstr); emit("goto _")
```

Precedence, from lowest to highest, is `||`, `&&`, `!`, the relational operators, `+ -`, `* /` and unary minus. Figure 2.2 combines all three operators with parentheses and a negation over a whole sub-expression, and puts an arithmetic expression with unary minus in the body.

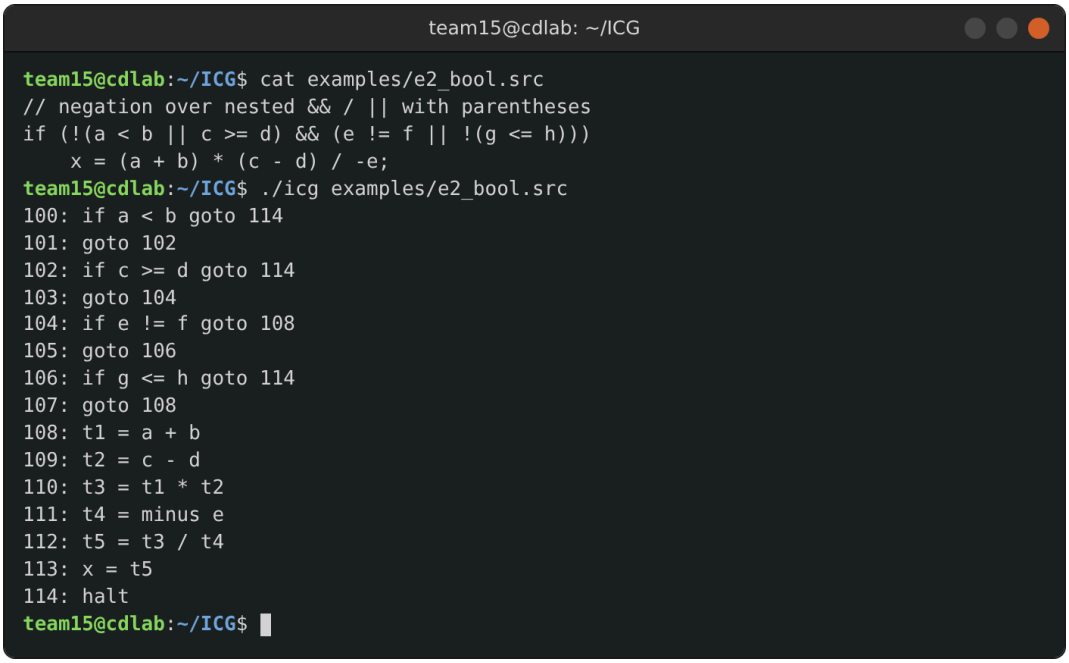


Figure 2.2 Negated disjunction inside a conjunction (`examples/e2_bool.src`)

The condition is `!(a < b || c >= d) && (…)`. Because of the `!`, the true exits of `a < b` and `c >= d` (quads 100 and 102) become false exits of the whole condition and jump to 114, past the assignment. Only when both tests fail does control reach 104, the start of the right operand of `&&`. There, a true `e != f` goes directly to the body at 108 without testing `g <= h`, and the inner `!` sends a true `g <= h` to the exit. In the body, `(a + b) * (c - d) / -e` is evaluated left to right with `t4 = minus e` computed before the division.

2.4 Nesting

Nesting needs no extra mechanism. Every rule reads only the lists of its own children and hands one `nextlist` up to its parent, so a construct behaves the same at any depth. A sequence of statements is joined by `L → L1 M S`, which fills the previous statement's `nextlist` with the first quad of the next statement. Two regression tests check this: `06_nested` (an `if-else` inside an `if-else` inside a `while`) and `15_dangling_else`, whose output is:

```
if (a < b) if (c < d) x = 1; else x = 2;

100: if a < b goto 102          104: x = 1
101: goto 107                   105: goto 107
102: if c < d goto 104          106: x = 2
103: goto 106                   107: halt
```

The `else` belongs to the inner `if`: a false `a < b` (101) skips everything, and only a false `c < d` (103) reaches `x = 2`.

2.5 Error reporting

Three kinds of error are detected, and each stops compilation with the line number, a message, a non-zero exit status, and no code output.

Table 2.1 Errors detected by the generator

Kind	Detected by	Message form	Exit
Lexical	Flex catch-all rule	`line N: unexpected character 'c'`	1
Syntax	Bison, `parse.error verbose`	`line N: syntax error, unexpected X, expecting Y`	1
Misplaced jump	Loop stack (Section 3.3)	`line N: break outside a loop`	2

Every token was given a readable name in the `%token` declarations (for example `RELOP "relational operator"`), so messages name the tokens in words instead of internal codes. Figure 2.3 shows one program for each case. In the first, the missing semicolon after `x = x + 1` is reported on line 4, where the parser meets the unexpected `}`; an LR parser can only detect a missing token at the next one it reads. In the second, the parser lists the three tokens that could legally follow `c > d`, which is enough to point to the missing parenthesis.

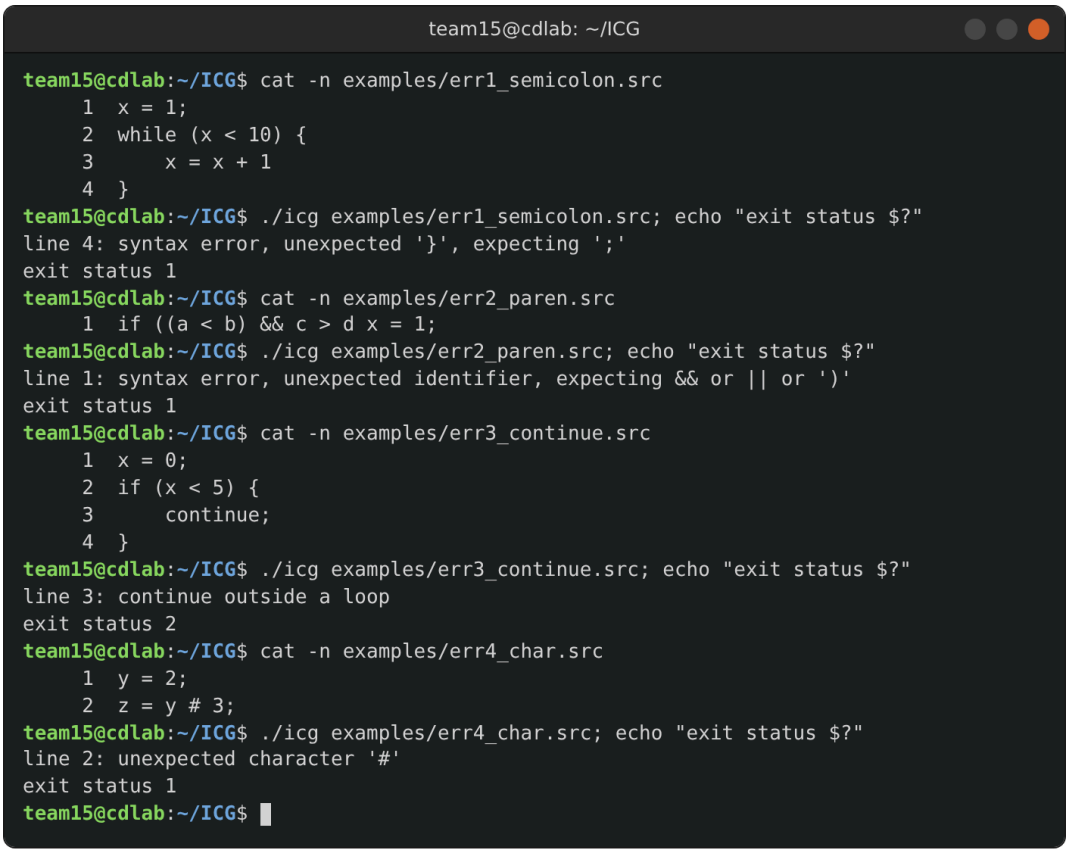


Figure 2.3 Syntax, semantic and lexical errors with line numbers and exit codes

2.6 Trace mode

The gap identified in the literature survey was that the patch lists of backpatching are invisible: a student sees finished quadruples, not the moment a jump was created empty and later filled. The `--trace` option addresses this. It prints every reduction that emits a jump or touches a patch list, followed by each operation that reduction performs, and then the finished code after a blank line. It is implemented by a few lines inside `emit`, `makelist`, `merge` and `backpatch`, and does not change the generated code.

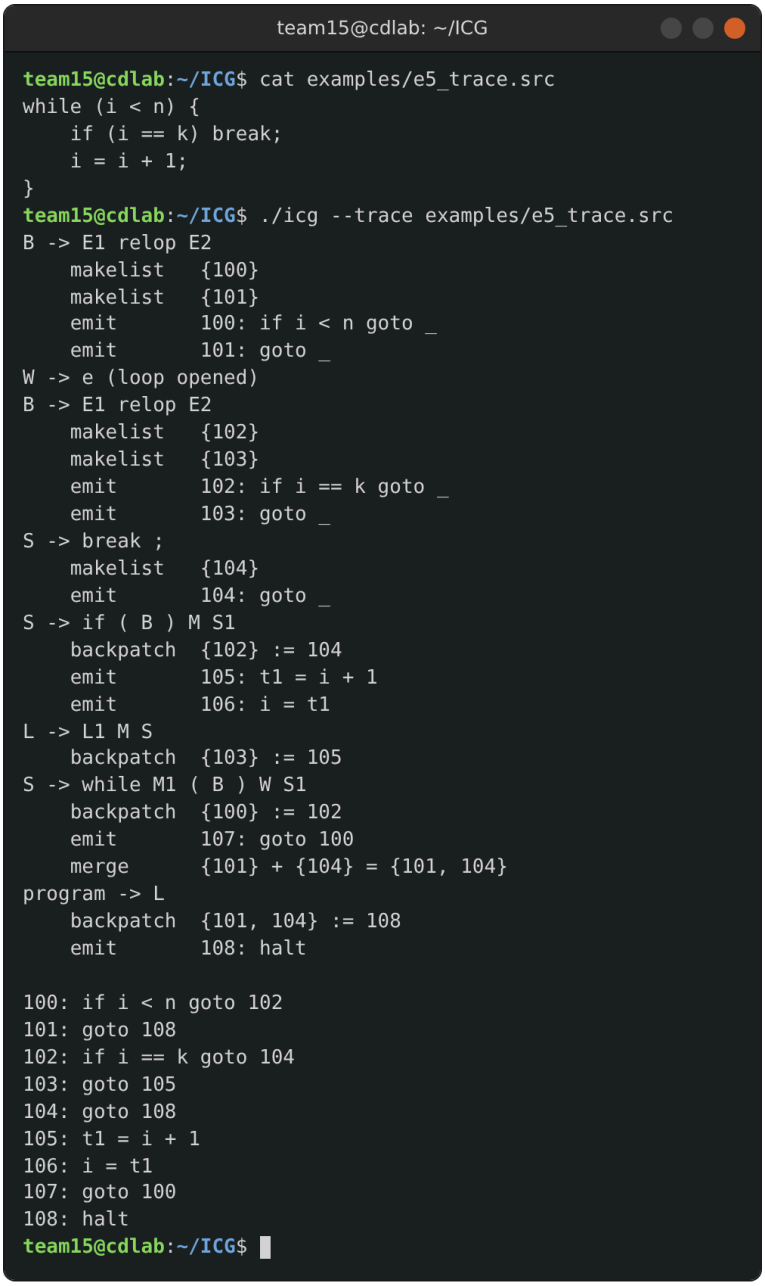


Figure 2.4 Trace of a while loop with a break (`examples/e5_trace.src`)

Reading Figure 2.4 from top to bottom gives the life of every jump. Table 2.2 collects it. Quad 104, the `break`, is the most interesting: it stays empty for the whole body, is merged with the loop's false exit 101 when the loop closes, and both are filled by the same `backpatch` at the end

of the program. Quad 107 never appears in a list, because its target was known when it was emitted.

Table 2.2 Life of each jump in Figure 2.4

Quad	Emitted by	Filled by	Target
100	B → E1 relop E2 (true exit)	while rule	102, loop body
101	B → E1 relop E2 (false exit)	program → L, after merge with 104	108, after the loop
102	B → E1 relop E2 (true exit)	if rule	104, the `break`
103	B → E1 relop E2 (false exit)	L → L1 M S	105, next statement
104	S → break ;	program → L, after merge with 101	108, after the loop
107	while rule (`emit_goto`)	known when emitted	100, loop test

2.7 Review 2 outcome

All Review 2 items were complete by the review date. The grammar still builds with no conflicts, and the regression suite grew from 3 tests to cover `if-else`, `while`, short-circuit conditions, constant conditions, nesting, the dangling else, lexical errors and trace output.

3 Review 3: Remaining Loops, Loop Exits and Comparison

Review 3 completed the language with `do-while`, `for`, `break` and `continue`, added the step-by-step mode and the remaining command-line outputs, and built a second, independent generator based on symbolic labels so that the backpatching generator could be checked against it and compared with it.

3.1 do-while

In a `do-while` loop the body comes first and the condition last, so the jump back to the start is a true exit of the condition rather than a separate `goto`. The marker `M` before the condition records where a `continue` must go: in this loop that is the condition, not the start of the body.

```
S → do W S1 while ( M B ) ;
      backpatch(S1.nextlist, M.instr)
      backpatch(B.truelist,  W.instr)          // repeat
      S.nextlist = merge(B.falselist, loop.breaks)
      continues  → M.instr
```

```
do {                            100: t1 = s + i
    s = s + i;                  101: s = t1
    i = i + 1;                  102: t2 = i + 1
} while (i <= n);               103: i = t2
t = s;                          104: if i <= n goto 100
                                105: goto 106
                                106: t = s
                                107: halt
```

3.2 for

The `for` loop is the one construct whose source order differs from its execution order: the step is written before the body but runs after it. Since a one-pass generator emits code in source order, the step is emitted before the body and the two are connected with jumps. The layout is init, test, step, jump to test, body, jump to step.

```
S → for ( A1 ; M1 B ; M2 A2 N ) W S1
      backpatch(B.truelist,  W.instr)          // test true  → body
      backpatch(N.nextlist,  M1.instr)         // after step → test
      backpatch(S1.nextlist, M2.instr)         // after body → step
      emit_goto(M2.instr)
      S.nextlist = merge(B.falselist, loop.breaks)
      continues  → M2.instr
```

Table 3.1 Layout of `for (i = 0; i < n; i = i + 1) s = s + i;` (test `09_for`)

Region	Quads	Code	Leaves to
init	100	`i = 0`	falls into test
test	101–102	`if i < n goto 106`, `goto 109`	body (106) or exit (109)
step	103–105	`t1 = i + 1`, `i = t1`, `goto 101`	test (101), quad 105 from `N`
body	106–108	`t2 = s + i`, `s = t2`, `goto 103`	step (103)
exit	109	`t = s`	next statement

3.3 break and continue

The textbook scheme has no attribute for a jump out of a loop, because the target of a `break` (the code after the loop) and of a `continue` (the test or the step) is not known where the statement is parsed, and the statement may be nested several levels inside `if`s. The generator keeps a stack of open loops for this purpose:

- `W` pushes an entry with two empty lists, `breaks` and `continues`, as the loop body begins.
- `break` and `continue` emit `goto _` and add it to the matching list of the innermost entry. With no open loop they stop with `line N: break outside a loop`.
- When the loop rule is reduced, `loop_end` pops the entry, fills the `continues` with the loop's continue target, and returns the `breaks`, which join the loop's `nextlist`.

Figure 3.1 is the hardest program in this report. It nests all three loop kinds, places a `continue` and a `break` in the innermost `for`, a second `break` in the middle `while`, and ends with a `do-while` condition that uses `&&` and `!`.

team15@cdlab: ~/ICG

```
team15@cdlab:~/ICG$ cat -n examples/e3_deep.src
     1  // three loop kinds nested, break and continue at different depths
     2  do {
     3      i = 0;
     4      while (i < n) {
     5          for (j = 0; j < m; j = j + 1) {
     6              if (a == j) continue;
     7              if (b == j && c > 0) break;
     8              s = s + i * j;
     9          }
    10          if (s > lim) break;
    11          i = i + 1;
    12      }
    13      k = k - 1;
    14  } while (k > 0 && !(s == lim));
    15  done = 1;
team15@cdlab:~/ICG$ ./icg examples/e3_deep.src | pr -2 -t -w 60
100: i = 0                    118: t3 = s + t2
101: if i < n goto 103        119: s = t3
102: goto 127                 120: goto 106
103: j = 0                    121: if s > lim goto 123
104: if j < m goto 109        122: goto 124
105: goto 121                 123: goto 127
106: t1 = j + 1               124: t4 = i + 1
107: j = t1                   125: i = t4
108: goto 104                 126: goto 101
109: if a == j goto 111       127: t5 = k - 1
110: goto 112                 128: k = t5
111: goto 106                 129: if k > 0 goto 131
112: if b == j goto 114       130: goto 133
113: goto 117                 131: if s == lim goto 133
114: if c > 0 goto 116        132: goto 100
115: goto 117                 133: done = 1
116: goto 121                 134: halt
117: t2 = i * j
team15@cdlab:~/ICG$
```

Figure 3.1 Three nested loop kinds with break and continue at different depths (`examples/e3_deep.src`)

Table 3.2 Loop exits in Figure 3.1, checked by hand

Quad	Source	Target	Why
111	`continue`, line 6	106	step `j = j + 1` of the innermost `for`
116	`break`, line 7	121	first quad after the `for`; leaves the `for` only
123	`break`, line 10	127	first quad after the `while`; leaves the `while` only
105	`for` test false	121	same target as the break at 116
102	`while` test false	127	same target as the break at 123
130, 131	`do-while` condition false	133	`k > 0` false, or `s == lim` true under `!`
132	`do-while` condition true	100	back to the start of the `do` body

Each `break` leaves exactly one loop, and each target agrees with what a C compiler would do. The loop stack, not the parse depth, decides the target: the `break` on line 7 sits inside an `if` inside a block inside the `for`, and still reaches the `for`'s entry because no other loop was opened in between.

3.4 Step mode and the remaining outputs

The Review 1 plan proposed a separate HTML viewer to replay the trace. It was replaced by `--step`, which prints the same trace but waits for Enter after each operation, so a class can follow one `makelist`, `merge` or `backpatch` at a time. Keystrokes are read from the terminal, so `--step` also works when the program itself is piped in on standard input. Three further options were completed in this phase:

Table 3.3 Command-line options of `icg`

Option	Output
`--format=tac`	Numbered three-address code (default)
`--format=quad`	Quadruple table `op, arg1, arg2, result`; jumps use `j<`, `j==`, ... and `j`, with the target in the result column
`--trace`, `--step`	Every rule and list operation, then the code; `--step` pauses after each
`--symbols`	Symbol table: each name and the line where it first appears
`--stats`	Quad count, peak number of unfilled jumps, patch-list entries (to stderr)

Figure 3.2 runs a bubble-sort skeleton with two nested `for` loops and an early-exit `break` through `--symbols` and `--format=quad`. Line numbers in the symbol table come from Bison location tracking rather than the lexer's current line, so each name is credited to the line it is written on (line 1 is a comment).

In the output, the `break` at 125 jumps to 127, the exit of the outer loop, joining the outer test's false exit at 103. Quad 126 is the jump from the end of the outer body back to the step; it cannot be reached because the `if` before it either breaks or jumps to the step itself (124). Unreachable and jump-to-next-quad instructions like 126 and 111 in Figure 2.1 are a known property of the textbook code shapes and would be removed by a peephole pass, which is outside the scope of this project.

```
team15@cdlab: ~/ICG
team15@cdlab:~/ICG$ cat examples/e4_sort.src
// bubble-sort style control flow on scalars
for (p = 0; p < n - 1; p = p + 1) {
    swapped = 0;
    for (q = 0; q < n - p - 1; q = q + 1) {
        if (x > y) { tmp = x; x = y; y = tmp; swapped = 1; }
    }
    if (swapped == 0) break;
}
team15@cdlab:~/ICG$ ./icg --symbols --format=quad examples/e4_sort.src
name         first line
p            2
n            2
swapped      3
q            4
x            5
y            5
tmp          5

#     op      arg1    arg2    result
100   =       0               p
101   -       n       1       t1
102   j<      p       t1      107
103   j                       127
104   +       p       1       t2
105   =       t2              p
106   j                       101
107   =       0               swapped
108   =       0               q
109   -       n       p       t3
110   -       t3      1       t4
111   j<      q       t4      116
112   j                       123
113   +       q       1       t5
114   =       t5              q
115   j                       109
116   j>      x       y       118
117   j                       113
118   =       x               tmp
119   =       y               x
120   =       tmp             y
121   =       1               swapped
122   j                       113
123   j==     swapped 0       125
124   j                       104
125   j                       127
126   j                       104
127   halt
team15@cdlab:~/ICG$
```

Figure 3.2 Symbol table and quadruple table for a bubble-sort skeleton (`examples/e4_sort.src`)

3.5 Two-pass baseline

To give the comparison promised in the literature survey a concrete basis, a second generator, `twopass`, was written as a separate Bison grammar (`src/twopass.y`, 501 lines) for the same language and sharing the same lexer. It follows the label-based approach of Appel [2]:

1. Pass 1 parses the program into a syntax tree.
2. Pass 2 walks the tree, giving each statement the label of the code that follows it and each condition the labels to jump to when true and false. Jumps name labels `L1`, `L2`, ...

3. Pass 3 replaces every label with the number of the quad it was placed before.

The code shapes were chosen to match `parser.y`, so the two programs must produce byte-identical output for any input. This makes `twopass` an independent check on every semantic action of the backpatching generator: a wrong `backpatch` in `icg` would show up as a difference. `--format=labels` prints the code after pass 2, before resolution (Figure 3.3).

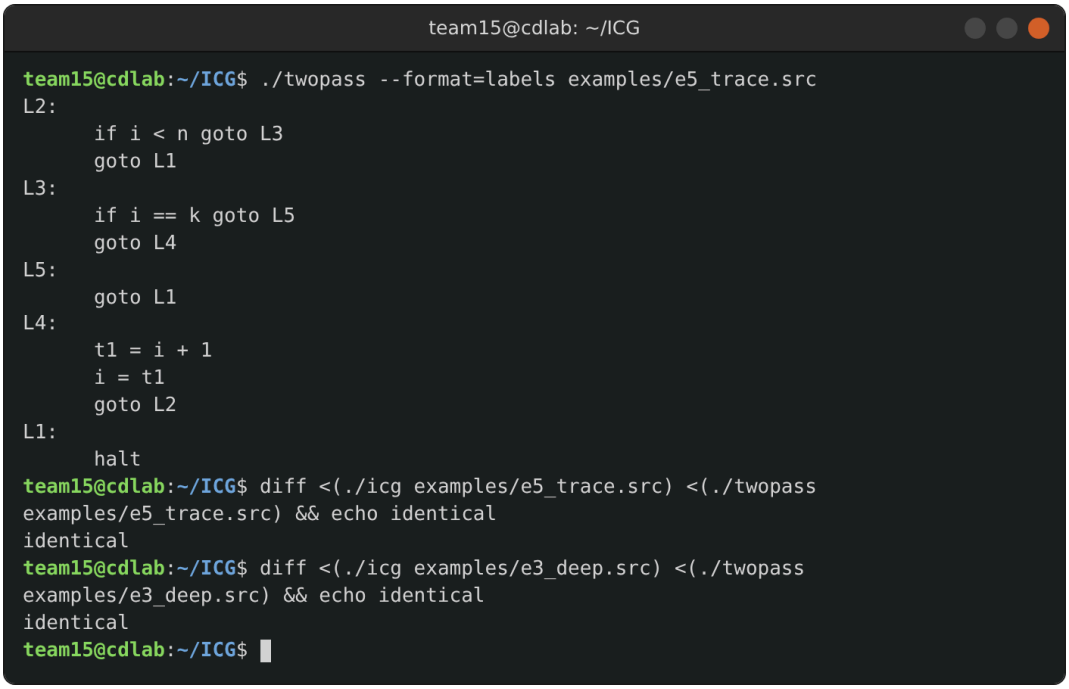


Figure 3.3 The baseline's code before label resolution, and diffs against `icg`

Comparing Figure 3.3 with the final code in Figure 2.4: `L2` resolves to 100, `L3` to 102, `L5` to 104, `L4` to 105 and `L1` to 108. The two `diff` commands print nothing and report `identical`, including for the triple-nested program of Figure 3.1.

3.6 Comparison of the two approaches

Both generators report statistics with `--stats`. For `icg` the important figure is the peak number of unfilled jumps: the most jumps waiting for a target at any one moment, which is everything the one-pass generator must hold beyond the code itself. For `twopass` the corresponding cost is the whole syntax tree, then the label table.

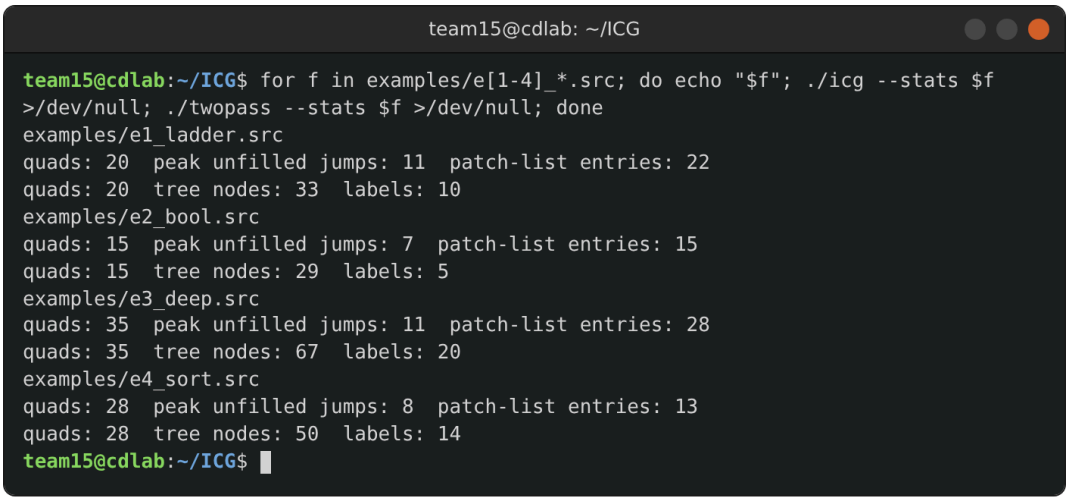


Figure 3.4 Statistics from both generators on the four stress programs

Table 3.4 icg versus twopass on regression tests and stress programs

Program	Quads	Peak unfilled jumps	Patch-list entries	Tree nodes	Labels
04_while	8	2	2	14	5
05_bool	8	5	10	15	4
06_nested	16	7	10	26	9
09_for	11	3	3	17	5
10_break_continue	17	8	11	29	9
14_nested_loops	18	7	12	32	11
e1_ladder	20	11	22	33	10
e2_bool	15	7	15	29	5
e3_deep	35	11	28	67	20
e4_sort	28	8	13	50	14

Both generators always produce the same number of quads. On the 35-quad program the backpatching generator never has more than 11 jumps pending, while the two-pass generator builds a 67-node tree before emitting anything. The tree grows with every expression and statement; the pending jumps grow only with the number of control-flow exits that are open at the same time. The else-if ladder shows the least favourable case for backpatching: 11 of its 20 quads are jumps that are pending together, because every branch's exit waits for the end of the ladder. The two-pass approach is easier to reason about, since every statement is handed its next label explicitly, but it needs the whole program in memory and an extra pass. Backpatching produces the same code in one pass, at the cost of the less obvious marker non-terminals, which the trace mode makes visible.

3.7 Documentation

The manuscript planned for Review 3 is complete and is kept in the repository's `docs/` folder: the project report (17 pages, `final_report.pdf` and `.docx`), the final review presentation (13 slides, `final_review.pptx`), and the literature survey. The `README` documents the language, every command-line option, the build on Linux and on Windows (`win_flex` / `win_bison` with `mingw32-make`), and the comparison with the baseline.

4 Testing and Validation

Testing was done at three levels: a regression suite with fixed expected outputs, a set of stress programs written for this report and checked by hand, and a randomised comparison of the two generators on generated programs.

4.1 Build

Both grammars are built with `%expect 0`, so Bison stops with an error if any shift/reduce or reduce/reduce conflict exists, and the generated C is compiled with `-Wall -Wextra`. Figure 4.1 is a clean build from source: no conflict reports and no compiler warnings.

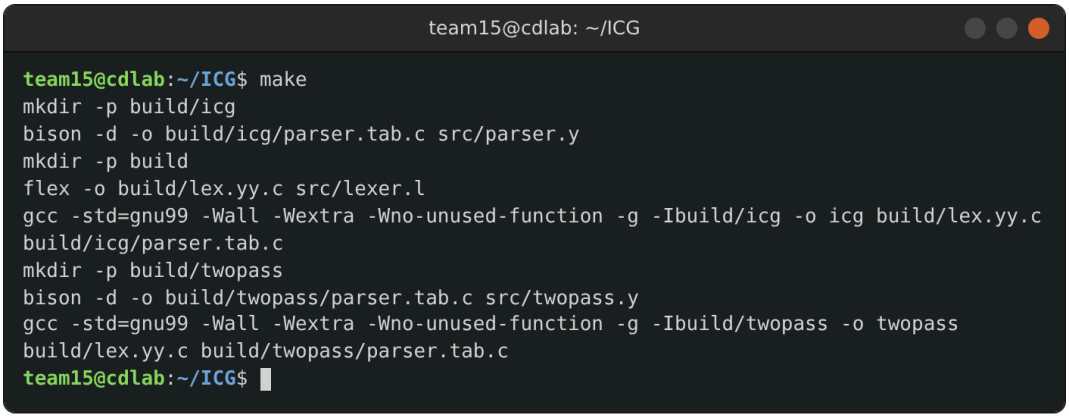


Figure 4.1 Clean build of both generators

4.2 Regression suite

The suite has 19 programs in `tests/`, each with a hand-checked `.expected` file holding the exact standard output and standard error. `run_tests.sh` compiles each program, diffs the output against the expected file, and fails any test whose final code still contains `goto _`, even if the output matches. Tests with an `.args` file pass options to `icg`; the other 16 are also run through `twopass` and must give identical output, giving 35 checks in total.

Table 4.1 Regression tests

Test	What it checks	Feature of	Result
`01_assign`	Arithmetic, precedence, unary minus, temporaries	Review 1	Pass
`02_if`	`if` without `else`	Review 1	Pass
`08_syntax_error`	Syntax error naming the expected tokens	Review 1	Pass
`17_quad_format`	`--format=quad` table	Review 1	Pass
`03_if_else`	`if-else`, the jump over the else-branch	Review 2	Pass
`04_while`	`while`, backward jump	Review 2	Pass
`05_bool`	`&&`, `||`, `!` with short-circuit jumps	Review 2	Pass
`06_nested`	`if-else` in `if-else` in `while`	Review 2	Pass
`11_true_false`	Constant conditions `true` and `false`	Review 2	Pass
`13_lex_error`	Unexpected character rejected	Review 2	Pass
`15_dangling_else`	`else` binds to the nearest `if`	Review 2	Pass
`16_trace`	`--trace` output for an `if-else`	Review 2	Pass
`07_do_while`	`do-while` loop	Review 3	Pass
`09_for`	`for`: init, test, step, body layout	Review 3	Pass
`10_break_continue`	`break` and `continue` in a `for`	Review 3	Pass
`12_break_outside`	`break` outside a loop rejected	Review 3	Pass
`14_nested_loops`	`break` in an inner loop, `continue` in the outer	Review 3	Pass
`18_symbols`	`--symbols`, names on later lines of a statement	Review 3	Pass
`19_error_line`	`break` and its `;` on different lines	Review 3	Pass

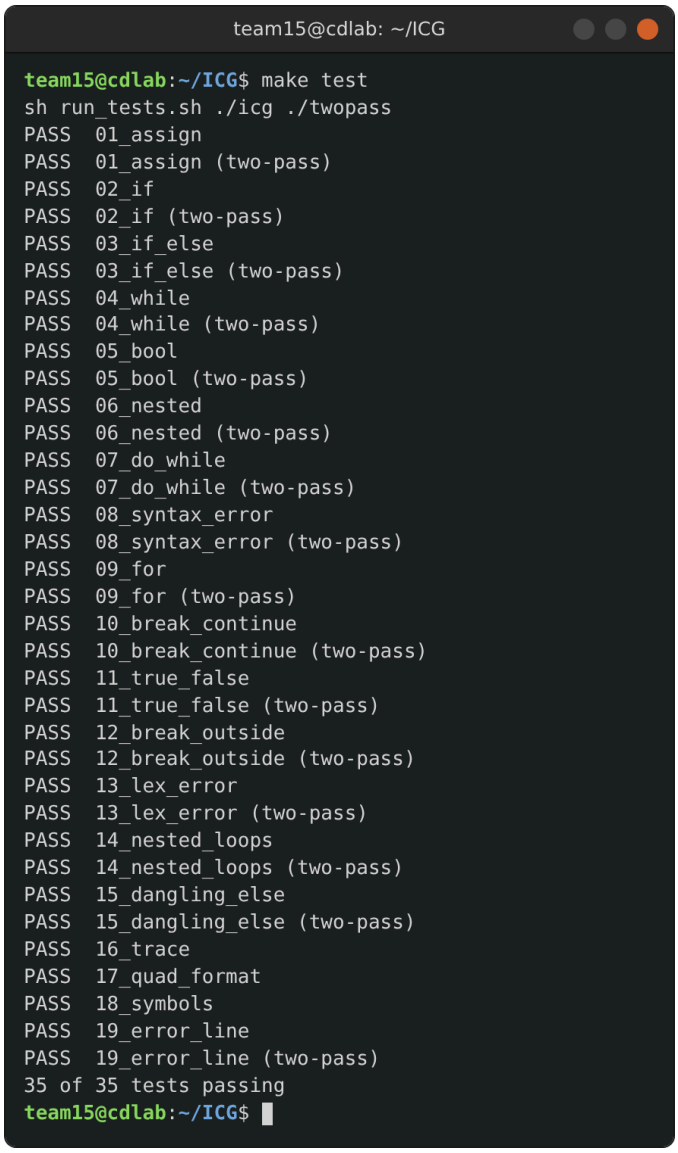


Figure 4.2 Full regression run: 35 of 35 checks pass

4.3 Stress programs

The regression tests each isolate one feature. The programs below were written for this report to combine features in ways the tests do not, and each output was traced by hand jump by jump before being accepted.

Table 4.2 Stress programs

Program	What makes it hard	Quads	Shown in	Result
`e1_ladder`	Four-level else-if chain; `&&` and `||` in different branches; three `N` jumps pending together	20	Figure 2.1	Correct
`e2_bool`	`!` over a parenthesised `||`, inside `&&`, with a second `!` nested in the right operand; unary minus in the body	15	Figure 2.2	Correct
`e3_deep`	`for` in `while` in `do-while`; `continue` and two `break`s at different depths; negated loop condition	35	Figure 3.1	Correct
`e4_sort`	Two nested `for` loops with computed bounds, a block body and an early-exit `break`	28	Figure 3.2	Correct
`e5_trace`	`break` merged with the loop's false exit; traced	9	Figure 2.4	Correct

All five also give identical output through `twopass`.

4.4 Error cases

Table 4.3 Error programs (Figure 2.3 and the regression suite)

Program	Fault	Reported
`err1_semicolon`	Missing `;` before `}`	`line 4: syntax error, unexpected '}', expecting ';'`
`err2_paren`	Unclosed `(` in a condition	`line 1: syntax error, unexpected identifier, expecting && or || or ')'`
`err3_continue`	`continue` in an `if`, no loop	`line 3: continue outside a loop`
`err4_char`	Character # not in the language	`line 2: unexpected character '#'`
`08_syntax_error`	`a < b < c`	`line 3: syntax error, unexpected relational operator, ...`
`19_error_line`	`break` on line 2, `;` on line 3	`line 2: break outside a loop`

4.5 Randomised comparison with the baseline

Hand-written tests only cover the cases someone thought of. As a final check, a small generator script produced random programs using every construct of the language: statements nested up to four levels, random arithmetic and boolean expressions with `!`, parentheses and constants, and `break` / `continue` placed only inside loops. Each program was compiled three ways, and three conditions were checked:

1. `icg` and `twopass` produce identical output and the same exit status;
2. the final code from `icg` contains no `goto _`;

3. the code printed after `icg --trace` is the same as without the trace.

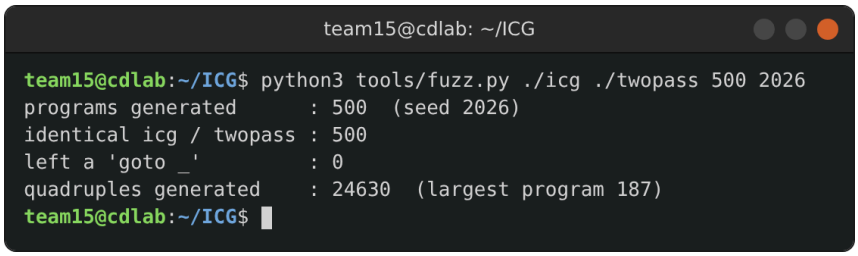


Figure 4.3 Randomised differential test on 500 programs

All 500 programs passed all three checks. Together they produced 24,630 quads, the largest program 187. Since the two generators were written independently, one with patch lists and one with labels, agreement on this many programs gives strong evidence that every semantic action of `icg` fills each jump with the right target.

5 Deviations from the Plan and Problems Solved

5.1 Trace viewer replaced by step mode

The Review 1 plan included an HTML viewer that would replay the trace in a browser. The plain-text trace already shows everything the viewer would: each list by its contents, and each jump from `goto _` to the `backpatch` that fills it. A separate viewer would have added a second language and a second program to keep in step with the generator. Instead, `--step` pauses the existing trace after every operation, which gives the same walkthrough inside the Flex and Bison program, in the lab terminal, with no extra files. The trace output is regular enough for a graphical viewer to be built on it later (Section 6.2).

5.2 Position of the N marker

With `N` placed before `else` as in the textbook, Bison reported a reduce/reduce conflict: after the then-branch, with `else` as lookahead, the parser could either reduce the else-less `if` or reduce the empty `N`. Precedence declarations only resolve shift/reduce conflicts, so the grammar was rewritten with `N` after `else` (Section 2.1). This does not change the generated code.

5.3 Exits from loops

The translation scheme in [1] covers `if` and `while` but has no attribute for `break` and `continue`. Passing a list down through every statement rule would have changed all of them. The loop stack opened by `W` (Section 3.3) keeps the change local to the loop rules and the two jump statements.

5.4 Line numbers for split statements

Error messages for `break` outside a loop initially used the lexer's current line. When the `;` was on the next line, the error was reported one line too late, because Bison only reduces `break ;` after reading the semicolon. Enabling `%locations` and using the line of the `break` token itself (`@1.first_line`) fixed this; test `19_error_line` guards it. The same change gave the symbol table exact first-use lines (test `18_symbols`).

5.5 Grammar warnings

Bison warned that some precedence declarations were never used for associativity. They were changed from `%nonassoc` / `%right` to `%precedence`, and an unused precedence for relational operators was removed. The generated parse tables are byte-identical before and after, and the build now produces no warnings.

5.6 Guard against wrong actions

`backpatch` checks that every quad on a list is a jump whose target is still empty. Filling a quad twice, or patching a non-jump, stops the generator with an internal error. A wrong semantic action therefore fails at once instead of producing plausible but wrong code. No test or random program triggers it in the finished generator.

6 Current Status and Remaining Work

6.1 Status

Table 6.1 Project status after Review 3 work

Measure	Value
Language constructs	Assignment, arithmetic, `if`, `if-else`, `while`, `do-while`, `for`, `break`, `continue`, blocks; `&&`, `||`, `!`, `true`, `false`
Source files	`lexer.l` (52 lines), `parser.y` (595), `twopass.y` (501)
Grammar conflicts	0 in both grammars (`%expect 0`)
Regression checks	35 of 35 pass (19 programs, 16 also through `twopass`)
Stress programs	5 of 5 correct, checked by hand
Randomised comparison	500 of 500 programs identical, no unfilled jumps
Objectives from Review 1	All four met: correct code, single pass, short-circuit booleans, error reporting

6.2 Remaining work and possible extensions

Implementation is complete. What remains is the Review 3 demonstration on 23 October and the upload of the final report and presentation to VTOP by 30 October. Natural extensions, outside the agreed scope, are:

- a `switch` statement, whose case exits fit directly into patch lists;
- arrays and procedure calls, adding address arithmetic and parameter passing to the three-address code;
- declarations and type checking in the symbol table;
- a peephole pass to remove jumps to the next quad and unreachable jumps such as 111 in Figure 2.1 and 126 in Figure 3.2;
- a graphical replay of the `--trace` output for classroom use.

References

1. A. V. Aho, M. S. Lam, R. Sethi, J. D. Ullman. *Compilers: Principles, Techniques, and Tools*, 2nd ed. Addison-Wesley, 2006. Ch. 6, sec. 6.6–6.7.
2. A. W. Appel. *Modern Compiler Implementation in C*. Cambridge University Press, 1998. Ch. 7.
3. K. D. Cooper, L. Torczon. *Engineering a Compiler*, 2nd ed. Morgan Kaufmann, 2011. Ch. 7.
4. J. Levine. *flex & bison*. O'Reilly Media, 2009.
5. C. Lattner, V. Adve. "LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation." *Proc. International Symposium on Code Generation and Optimization (CGO)*, 2004, pp. 75–86.
6. S. H. Rodger, T. W. Finley. *JFLAP: An Interactive Formal Languages and Automata Package*. Jones and Bartlett, 2006.

Appendix A. Reproducing the Results

Requirements: gcc, Flex and Bison. The commands below reproduce every figure in this report from the project directory.

```
make                                          # Figure 4.1
make test                                     # Figure 4.2
./icg examples/e1_ladder.src                  # Figure 2.1
./icg examples/e2_bool.src                    # Figure 2.2
./icg examples/err1_semicolon.src             # Figure 2.3 (and err2..err4)
./icg --trace examples/e5_trace.src           # Figure 2.4
./icg examples/e3_deep.src                    # Figure 3.1
./icg --symbols --format=quad examples/e4_sort.src    # Figure 3.2
./twopass --format=labels examples/e5_trace.src       # Figure 3.3
./icg --stats examples/e3_deep.src            # Figure 3.4
./twopass --stats examples/e3_deep.src
python3 tools/fuzz.py ./icg ./twopass 500 2026 # Figure 4.3
./icg --step tests/03_if_else.src             # step mode, Enter to advance
```